Replication of Baumann et al. (2020) in R

This notebook replicates Figs. 1 and 2 of Baumann et al. (2020), *Modeling Echo Chambers and Polarization Dynamics in Social Networks*, PRL 124, 048301. We follow the algorithm described in the Supplementary Material (SM), section "Numerical simulations".

It is a stand-alone version of the model in plain R, independent of the NetLogo model in `netlogo/polarisation.nlogox`. It needs no input files; the figures are shown here and also saved to `results/replication/`.

```
output_folder <- "../results/replication"
dir.create(output_folder, recursive = TRUE, showWarnings = FALSE)
```

The model

There are N agents. Each agent i has an opinion x_i: the sign says which side the agent is on, the size says how strongly. Opinions change according to Eq. (1) of the paper:

$$\dot{x}_i = -x_i + K \sum_{j=1}^{N} A_{ij}(t) \tanh(\alpha x_j)$$

- $-x_i$: without any contacts, an opinion slowly fades back to 0 (neutral).
- $A_{ij}(t) = 1$ if agent j influences agent i at time t, otherwise 0.
- $\tanh(\alpha x_j)$: how strongly agent j pushes others towards its own side. It grows with j's conviction but never exceeds 1. The larger α (how controversial the topic is), the faster it grows.
- K: overall strength of social influence.

Who talks to whom ($A_{ij}(t)$) changes every time step. Each agent has an *activity* a_i, the probability of starting conversations in a time step. Active agents prefer partners with similar opinions (*homophily*, controlled by β).

The parameters, with the values used in the paper:

Parameter	Meaning	Value
N	number of agents	1000
α	controversialness of the topic	varies
β	homophily (0 = partners chosen at random)	varies
K	social influence strength	3 (Fig. 1)
r	probability that a contacted agent answers back	0.5
m	number of agents an active agent contacts	10
ϵ	smallest possible activity	0.01
γ	exponent of the activity distribution	2.1
dt	length of one time step	0.01
steps	number of time steps (1000 steps = time 10)	1000

Building blocks

Activities

Each agent gets an activity between ϵ and 1 at the start and keeps it. Activities follow a power law $F(a) \propto a^{-\gamma}$: most agents are rarely active, a few are very active.

To draw from this distribution we use *inverse transform sampling*: draw a uniform random number u between 0 and 1 and plug it into the inverse of the cumulative distribution, which gives

$$a = \left(\epsilon^{1-\gamma} + u\left(1 - \epsilon^{1-\gamma}\right)\right)^{1/(1-\gamma)}$$

```
sample_activities <- function(N, epsilon, gamma) {
  u <- runif(N)
  (epsilon^(1 - gamma) + u * (1 - epsilon^(1 - gamma)))^(1 / (1 - gamma))
}
```

Choosing partners

An active agent i contacts m different agents. Agent j is picked with a probability that shrinks with the opinion distance, Eq. (2):

$$p_{ij} = \frac{|x_i - x_j|^{-\beta}}{\sum_j |x_i - x_j|^{-\beta}}$$

With $\beta = 0$ every agent is equally likely. The larger β, the more agent i sticks to agents with similar opinions.

```
contact_prob <- function(i, x, beta) {
  distance <- abs(x[i] - x)
  weight <- distance^(-beta)
  weight[i] <- 0 # agents don't contact themselves
  weight[is.infinite(weight)] <- 1e12 # two identical opinions would give 1/0 = Inf
  weight / sum(weight) # turn weights into probabilities
}
```

Updating the opinions

`opinion_change()` is the right-hand side of Eq. (1): how fast each opinion changes right now. `A %*% tanh(alpha * x)` is a matrix product that computes $\sum_j A_{ij} \tanh(\alpha x_j)$ for all agents i at once.

To move the opinions one time step forward, the SM uses the fourth-order Runge-Kutta method (RK4). The simplest method (Euler) would just do `x + dt * opinion_change(x)`, using the rate of change at the start of the step. RK4 instead estimates the rate of change four times (at the start, twice halfway, and at the end of the step) and takes a weighted average, which is much more accurate. The network `A` stays the same during the whole step.

```
opinion_change <- function(x, A, K, alpha) {
  -x + K * as.vector(A %*% tanh(alpha * x))
}

rk4_step <- function(x, A, K, alpha, dt) {
  k1 <- opinion_change(x, A, K, alpha)
```

```
  k2 <- opinion_change(x + dt / 2 * k1, A, K, alpha)
  k3 <- opinion_change(x + dt / 2 * k2, A, K, alpha)
  k4 <- opinion_change(x + dt * k3, A, K, alpha)
  x + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
}
```

The simulation

Opinions start evenly spread between -1 and 1. Then, in every time step (SM, "Numerical simulations"):

1. Each agent i becomes active with probability a_i.
2. Each active agent i contacts m agents j, chosen as above. Those agents are now influenced by i: $A_{ji} = 1$.
3. Each contacted agent j answers back with probability r, so it also influences i: $A_{ij} = 1$.
4. All opinions are updated with one RK4 step.
5. The network is thrown away; the next step starts with a new one.

`simulate()` returns the opinions of all agents at all time steps (one row per time step, one column per agent), together with the parameters it used, so that the plots can show them.

```
simulate <- function(alpha, beta, K = 3, r = 0.5, N = 1000, m = 10,
                     epsilon = 0.01, gamma = 2.1, dt = 0.01, n_steps = 1000) {
  activities <- sample_activities(N, epsilon, gamma)
  x <- seq(-1, 1, length.out = N)

  opinions <- matrix(NA, nrow = n_steps + 1, ncol = N)
  opinions[1, ] <- x

  for (step in 1:n_steps) {
    A <- matrix(0, nrow = N, ncol = N) # 5. a new, empty network every step

    active <- which(runif(N) < activities) # 1. who is active?

    for (i in active) {
      # 2. i contacts m different agents, preferring similar opinions
      contacts <- sample(1:N, size = m, prob = contact_prob(i, x, beta))
      A[contacts, i] <- 1

      # 3. each contact answers back with probability r
      answers <- contacts[runif(m) < r]
      A[i, answers] <- 1
    }

    x <- rk4_step(x, A, K, alpha, dt) # 4. update the opinions
    opinions[step + 1, ] <- x
  }

  list(
    time = (0:n_steps) * dt, opinions = opinions,
    alpha = alpha, beta = beta, K = K, r = r, N = N
  )
}
```

Figure 1: three kinds of outcome

The plot function draws every agent's opinion over time, blue while it is positive and red while it is negative, as in the paper. It does this by drawing the positive and the negative values separately.

```
plot_opinions <- function(sim, title) {
  positive <- sim$opinions
  positive[positive <= 0] <- NA
  negative <- sim$opinions
  negative[negative > 0] <- NA

  matplot(sim$time, positive,
    type = "l", lty = 1, col = "blue",
    ylim = range(sim$opinions), xlab = "t", ylab = "opinion x",
    main = title
  )
  matlines(sim$time, negative, lty = 1, col = "red")

  subtitle <- paste0(
    "alpha = ", sim$alpha, ", beta = ", sim$beta,
    ", K = ", sim$K, ", r = ", sim$r, ", N = ", sim$N
  )
  mtext(subtitle, side = 3, line = 0.3)
}
```

All three panels use $K = 3$ and $r = 0.5$; only α and β differ. Each simulation takes a few seconds.

Fig. 1a: neutral consensus ($\alpha = 0.05$, $\beta = 2$)

The topic is hardly controversial, so agents barely influence each other. Every opinion fades back to 0.

```
set.seed(31415)
sim_1a <- simulate(alpha = 0.05, beta = 2)
file <- file.path(output_folder, "fig1a-consensus.png")
png(file, width = 1000, height = 700, res = 120)
plot_opinions(sim_1a, "Fig. 1a: neutral consensus")
dev.off()
```

```
## pdf
##   2
```

```
knitr::include_graphics(file)
```

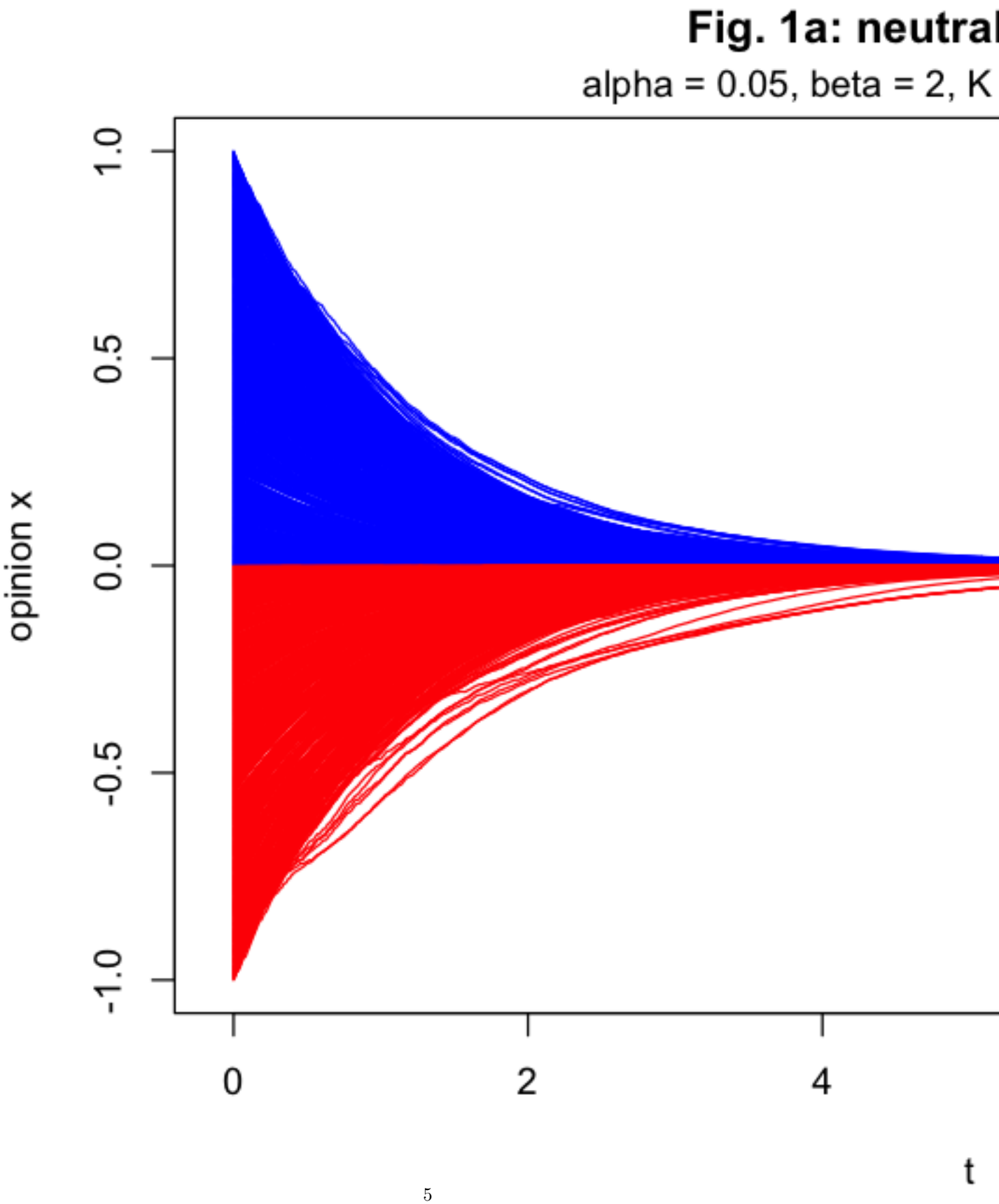

Fig. 1a: neutral
alpha = 0.05, beta = 2, K
opinion x
1.0
0.5
0.0
-0.5
-1.0
0
2
4
t

Fig. 1b: one-sided radicalization ($\alpha = 3$, $\beta = 0$)

Agents pick partners at random, so both sides keep talking to each other, and one side ends up absorbing everyone. Which side wins is down to chance, so the opinions may end up all negative instead of all positive as in the paper. Both are correct.

```
set.seed(31416)
sim_1b <- simulate(alpha = 3, beta = 0)
file <- file.path(output_folder, "fig1b-one-sided-radicalization.png")
png(file, width = 1000, height = 700, res = 120)
plot_opinions(sim_1b, "Fig. 1b: one-sided radicalization")
dev.off()
```

```
## pdf
##   2
```

```
knitr::include_graphics(file)
```

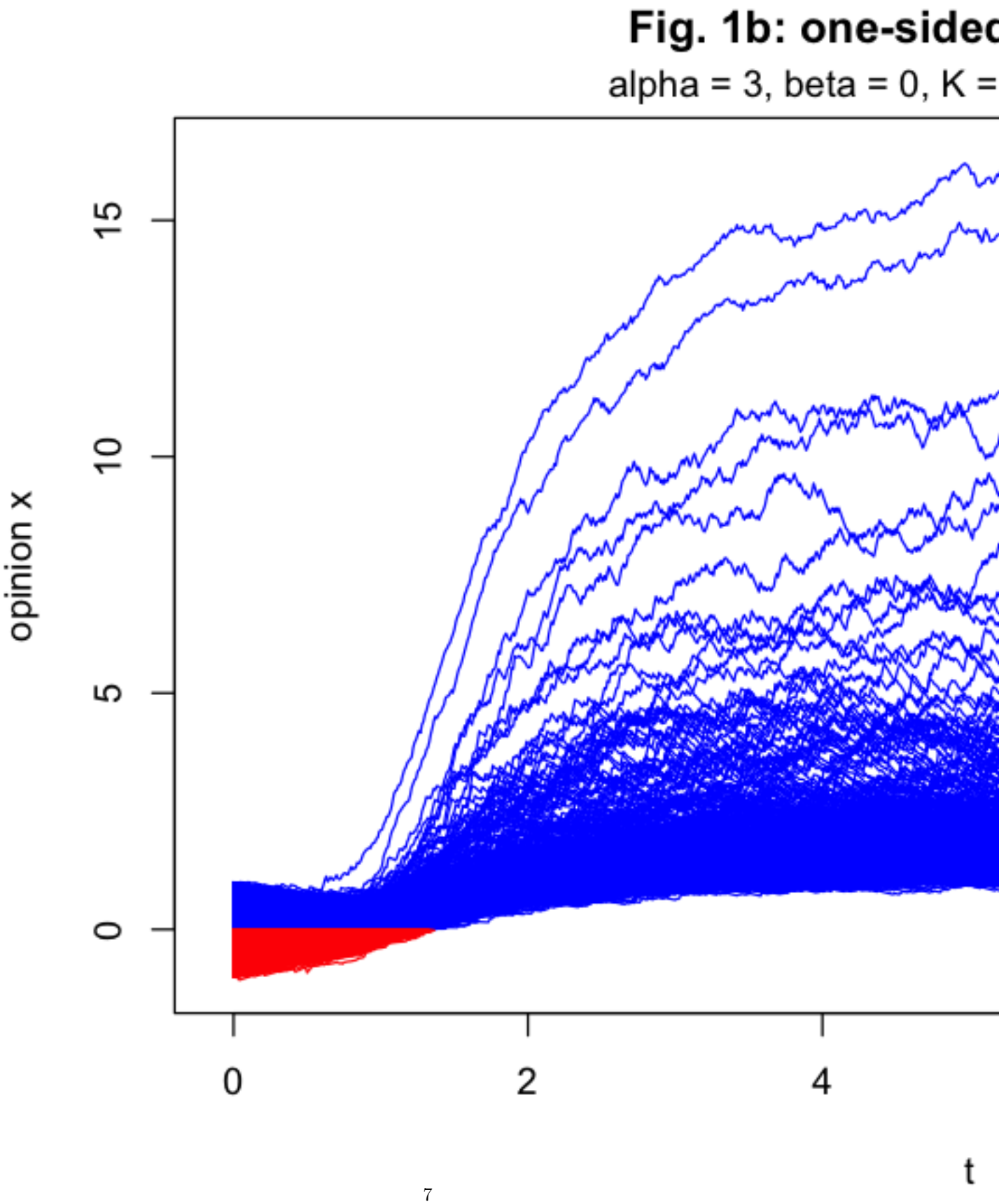

Fig. 1b: one-sided
alpha = 3, beta = 0, K =
opinion x
0
5
10
15
0
2
4
t

Fig. 1c: polarization ($\alpha = 3$, $\beta = 3$)

Agents mostly talk to like-minded agents, so each side reinforces itself and the population splits into two groups: echo chambers.

```
set.seed(31417)
sim_1c <- simulate(alpha = 3, beta = 3)
file <- file.path(output_folder, "fig1c-polarization.png")
png(file, width = 1000, height = 700, res = 120)
plot_opinions(sim_1c, "Fig. 1c: opinion polarization")
dev.off()
```

```
## pdf
##   2
```

```
knitr::include_graphics(file)
```

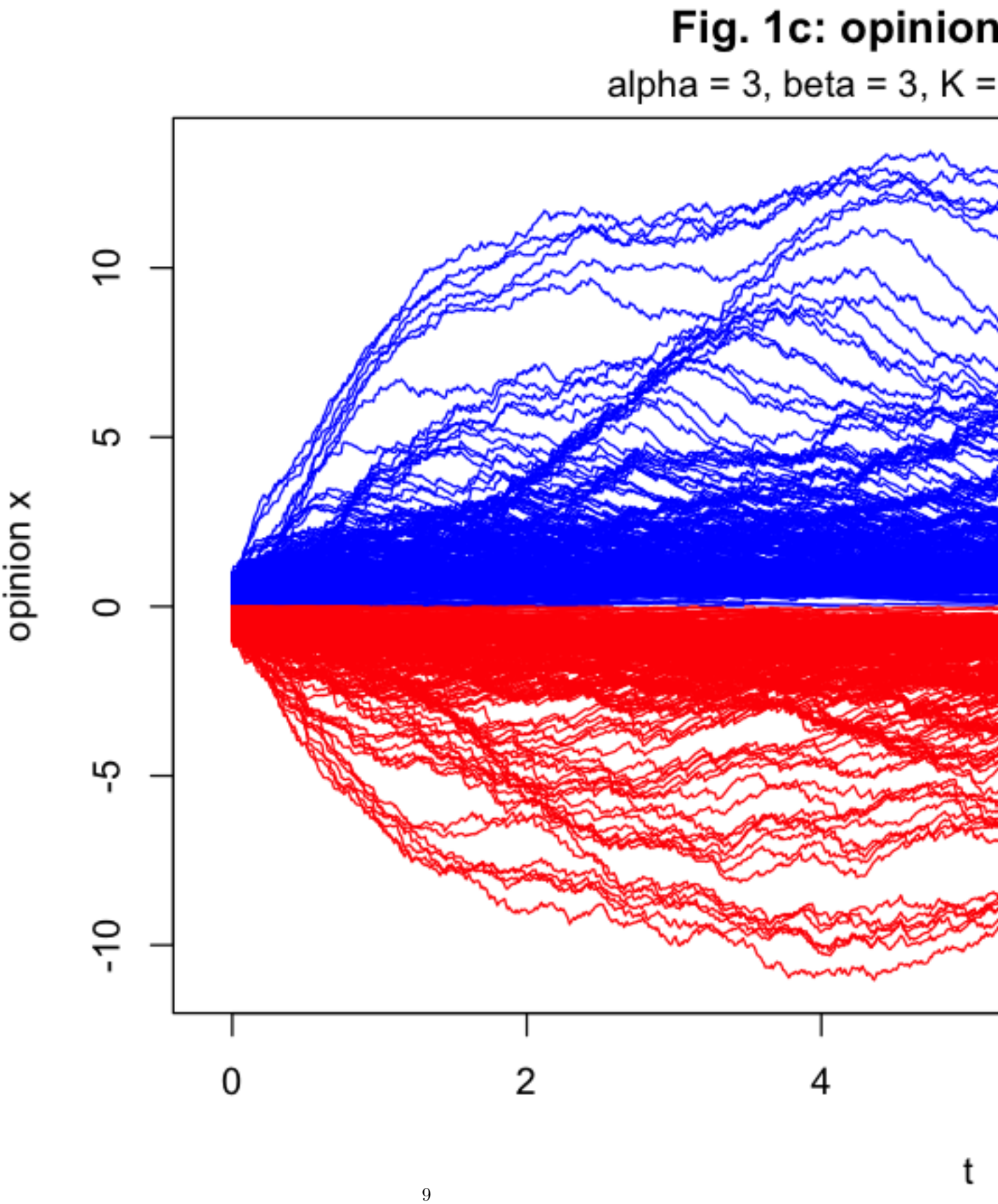

Fig. 1c: opinion
alpha = 3, beta = 3, K =
opinion x
10
5
0
-5
-10
0
2
4
t

Figure 2: from consensus to radicalization

Fig. 2 runs the model for many combinations of K and α (with $\beta = 0.5$ and $r = 0.5$) and shows, for each combination, how far the average opinion at the end is from 0:

$$|\langle x_f \rangle| = \left| \frac{1}{N} \sum_i x_i(t_{\text{final}}) \right|$$

A value of 0 means the population reached a neutral consensus; a large value means it radicalized.

The paper also derives where the switch should happen (Eq. 3): the consensus breaks down once α is larger than

$$\alpha_c = \frac{1}{(1+r)\, K\, m\, \langle a \rangle}$$

where $\langle a \rangle$ is the average activity. For our activity distribution it is (SM)

$$\langle a \rangle = \frac{1-\gamma}{2-\gamma} \cdot \frac{1-\epsilon^{2-\gamma}}{1-\epsilon^{1-\gamma}} \approx 0.041$$

We run each combination once, up to $t = 10$ as in Fig. 1 (the paper doesn't say how long they ran it).

The paper uses a grid of about 40 x 40. We use 20 x 20, which is 400 simulations. That takes a while (about 40 minutes), so the results are cached: knitting again reuses them as long as this chunk doesn't change. Delete the `replication_cache` folder after changing the model. For a quick look, use `by = 0.4`.

```
beta_fig2 <- 0.5
r_fig2 <- 0.5
alpha_values <- seq(0.1, 3.9, by = 0.2)
K_values <- seq(0.1, 3.9, by = 0.2)

# one row per alpha value, one column per K value
final_opinion <- matrix(NA, nrow = length(alpha_values), ncol = length(K_values))

set.seed(31415)
for (a in seq_along(alpha_values)) {
  for (k in seq_along(K_values)) {
    sim <- simulate(
      alpha = alpha_values[a], beta = beta_fig2,
      K = K_values[k], r = r_fig2
    )
    last_step <- sim$opinions[nrow(sim$opinions), ]
    final_opinion[a, k] <- abs(mean(last_step))
  }
}
```

The heat map shows $|\langle x_f \rangle|$: dark is consensus, bright is radicalization. The dashed white line is the prediction α_c.

```
file <- file.path(output_folder, "fig2-phase-diagram.png")
png(file, width = 900, height = 800, res = 120)
```

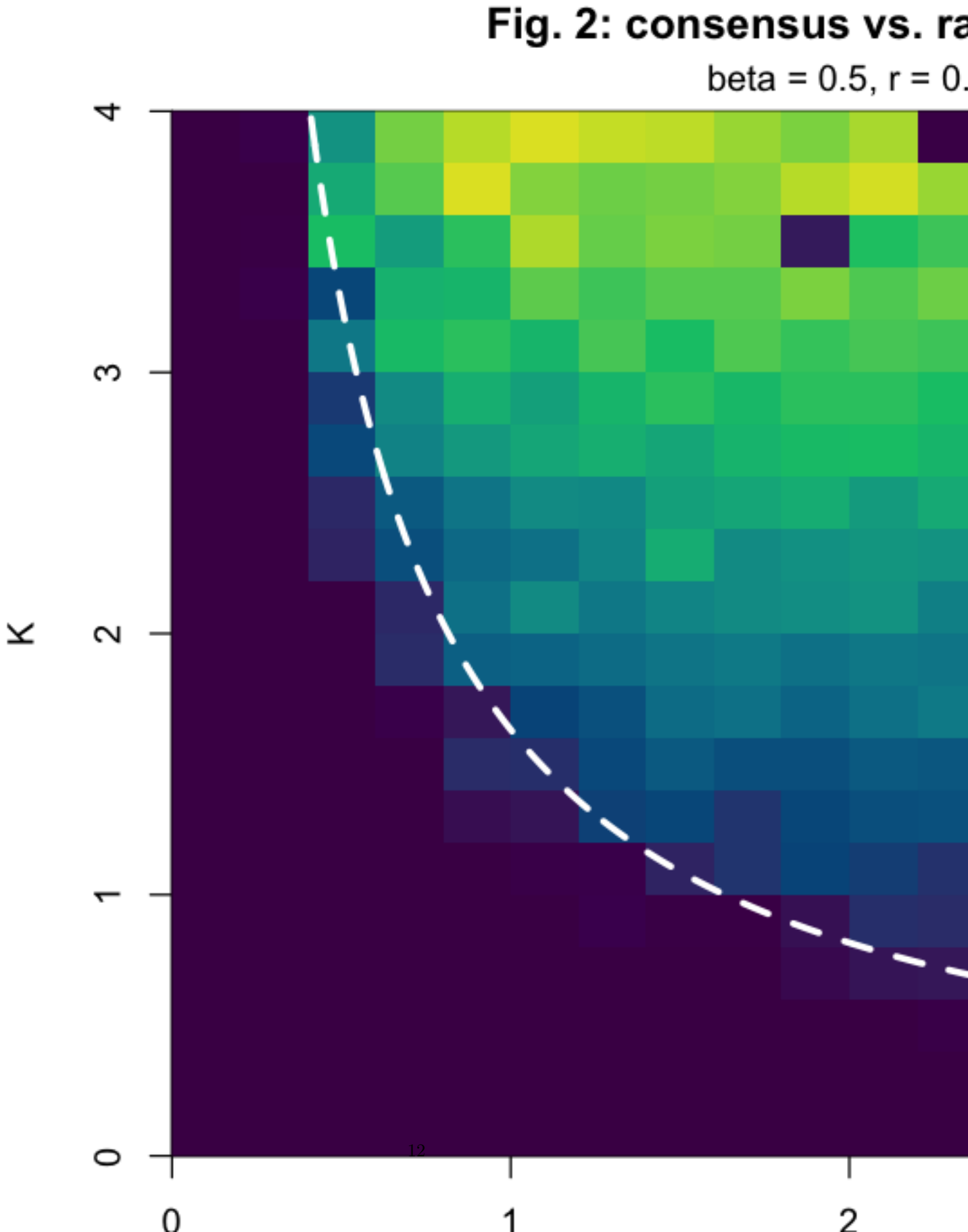
```r
image(alpha_values, K_values, final_opinion,
  col = hcl.colors(100, "viridis"),
  xlab = "alpha", ylab = "K",
  main = "Fig. 2: consensus vs. radicalization"
)
mtext(paste0("beta = ", beta_fig2, ", r = ", r_fig2), side = 3, line = 0.3)

# prediction alpha_c for every K, using the paper's m, epsilon and gamma
m <- 10
epsilon <- 0.01
gamma <- 2.1
mean_a <- (1 - gamma) / (2 - gamma) * (1 - epsilon^(2 - gamma)) / (1 - epsilon^(1 - gamma))
K_line <- seq(0.01, 4, by = 0.01) # many K values, for a smooth line
alpha_c <- 1 / ((1 + r_fig2) * K_line * m * mean_a)
lines(alpha_c, K_line, lty = 2, lwd = 3, col = "white")

dev.off()
```

```
## pdf
##   2
```

```r
knitr::include_graphics(file)
```


Fig. 2: consensus vs. ra
beta = 0.5, r = 0.
K
0
1
2
3
4
0
1
2